

BEAM Oracle Adapters Audit Report

Table of Contents

1. Executive Summary

4

About Loopscale BEAM Oracle Adapters

4

Audit Summary

4

Overall Security Posture

4

Before Fix Review Summary

5

After Fix Review Summary

5

Launch Recommendations

5

Audit Scope

5

2. Assumptions and Considerations

6

Audit Assumptions

6

Considerations

6

Trust Assumptions

6

3. Severity Definitions

7

Impact

7

Likelihood

7

Severity Classification Matrix

7

4. Findings

9

M01: BEAM-backed Pyth prices ignore Loopscale’s EMA price policy

9

M02: Pyth Pro prices are not bound to an expected feed ID

12

M03: Chainlink V10 rejects normal no-action reports as pending corporate actions

14

L01: Unbounded negative Linear configurations can permanently disable the feed

16

L02: Exponent vault asset binding is only checked at configuration time

18

L03: Chainlink V10 freshness uses report validity time instead of the last price update time

20

About Us

22

About Adevar Labs22

Audit Methodology22

Confidentiality Notice24

Legal Disclaimer24

1. Executive Summary

About Loopscale BEAM Oracle Adapters

Loopscale BEAM is Loopscale's in-house oracle aggregation and caching layer. BEAM stores normalized price entries that can later be consumed by Loopscale as oracle inputs for borrowing, collateral valuation, liquidation, and other risk-sensitive flows.

The reviewed pull request extends BEAM's oracle capabilities by adding support for additional data sources and price models, including Chainlink Data Streams V9 and V10 reports, Pyth Pro-compatible price update accounts, a synthetic linear price model, and an Exponent discount-to-maturity model. These additions are intended to support a broader set of assets, including NAV-style feeds, tokenized equity/RWA feeds, synthetic assets, and maturity-based pricing.

Audit Summary

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	3	3	0
Low	3	3	0

Enhancement Opportunities: 0

Overall Security Posture

The reviewed changes are generally structured around explicit provider adapters with clear validation boundaries. The implementation includes defensive checks such as Chainlink verifier integration, report feed-id checks for Chainlink reports, market-status checks for Chainlink V10, ripcord handling for Chainlink V9, full-verification enforcement for Pyth price update accounts, and owner checks on external oracle accounts.

The main security concerns are concentrated around integration semantics rather than basic account validation. Several findings involve cases where provider-specific assumptions are compressed into generic BEAM price entries, or where new BEAM sources are consumed by existing Loopscale oracle paths with slightly different expectations. The highest-priority issues are incorrect handling of Chainlink

V10 corporate-action sentinel values, missing Pyth feed-id binding for Pyth Pro-compatible accounts, and mismatched spot/EMA handling for BEAM-backed Pyth prices.

Before Fix Review Summary

The review identified six findings across the BEAM oracle changes: three Medium-risk findings and three Low-risk findings.

The main areas of concern were oracle-source identity, Chainlink V10 report semantics, downstream Loopscale handling of BEAM-normalized prices, and edge cases in the new model-based pricing paths. Overall, the issues are addressable and mostly relate to integration correctness rather than broad access-control failures.

Two additional behaviors were documented as considerations rather than findings: V10 cached-price freshness during market closures and the bounded rational-interpolation rounding policy.

After Fix Review Summary

The after-fix review found that all six findings were remediated.

The remediation added feed-id binding for Pyth/Pyth Pro BEAM prices, fixed Chainlink V10 corporate-action and freshness handling, threaded Loopscale's **PriceType** policy through BEAM consumption, rejected unbounded negative Linear configurations, and revalidated the Exponent PT mint binding during price reads.

The remaining follow-up is integration hygiene rather than an open security finding: generated artifacts such as the BEAM IDL/types should be regenerated to include the new **AdditionalOracleParams::Pyth** variant.

Launch Recommendations

Before launch, we recommend completing final integration hygiene checks, including regenerating generated artifacts such as the BEAM IDL/types, double-checking timestamp and decimal units across new adapters, and keeping focused regression tests for each remediated finding.

Audit Scope

- **Repository:** <https://github.com/LoopscaleLabs/loopscale-program-library>
- **Base Commit Hash:** **5bd2807ddddf3200f5403021e3a1f2819c130200**
- **Before Fix Review Commit Hash:** **8e33f14e1cdec5570f90d63e270a8b87b10fcc9c**
- **After Fix Review Commit Hash:** **0e35807e6f5b69a75a489c722c9dd6c4e395db80**
- **Files/Modules in Scope:** Changes made to **programs/beam** between base commit and top commit

2. Assumptions and Considerations

Audit Assumptions

I. PR-scoped review: The assessment focused on the BEAM changes introduced between base commit `5bd2807dddf3200f5403021e3a1f2819c130200` and top commit `8e33f14e1cdec5570f90d63e270a8b87b10fcc9c`. Adjacent LoopScale code was reviewed only where needed to understand how BEAM prices are consumed.

II. External oracle correctness: Chainlink Data Streams, Pyth/Pyth Pro verification, and Exponent account decoding are assumed to behave according to their documented schemas and security guarantees.

III. Exponent PT pricing: In order to protect against price manipulations, the `get_discount_to_maturity` function uses a linear approximation to compute the PT exchange rate, which can differ slightly from the actual exchange rate computed by Exponent.

Considerations

I. V10 cached-price freshness during market closures: Chainlink V10 equity/RWA reports are accepted only while `market_status == Open`; after a valid open-market report is cached, downstream consumption follows LoopScale's normal BEAM freshness model using the configured `max_age`. With the current `u16` freshness field, the maximum expressible window is 65,535 seconds, about 18.2 hours, so weekend availability for last-close pricing requires either a wider freshness field or operationally ensuring those assets are not configured for weekend use until the newer version is deployed.

II. Rational interpolation rounding: The shared rational interpolation helper uses Rust integer division, which truncates toward zero. For the discount-to-maturity path, this can round negative fractional slope terms upward by less than one 18-decimal unit. This is treated as accepted precision behavior because the bound is extremely small, the direction is documented, and changing it would alter existing pinned canonical values.

Trust Assumptions

Provider-specific invariants outside BEAM are assumed to remain true after configuration. For example, Exponent vault fields used for pricing are assumed to remain immutable unless BEAM explicitly revalidates them during price updates.

BEAM-normalized prices are assumed to be consumed according to the current LoopScale oracle implementation: cached prices remain valid until `max_age` expires, consumers read `BeamPriceEntry`, and provider-specific state such as Chainlink market status is not rechecked after the BEAM cache entry is written.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

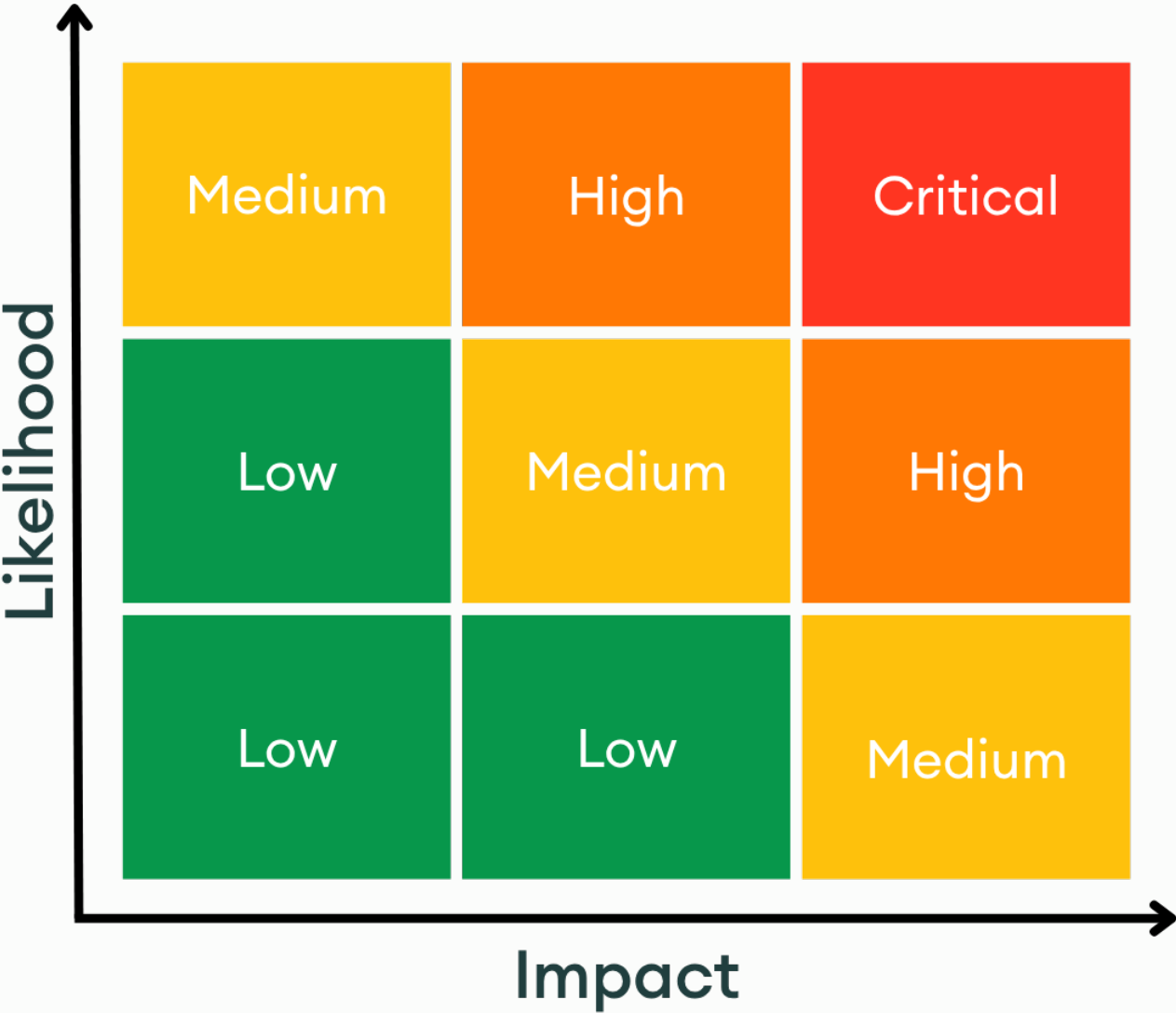
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.